

PDF Tool

Technical Documentation

Privacy-first online PDF editor
Live: <https://pdftool4u.duckdns.org>
Guide: <https://pdftool4u.duckdns.org/help>
Repository: github.com/mukeshroyhub/pdf_tool

Edition: July 2026 (post privacy-architecture release)
Owner: Mukesh Kumar

1. Overview

PDF Tool is a free, full-featured online PDF editor with a distinctive privacy model: **user files are stored only in the user's browser** (IndexedDB), never on the server. Server-side tools process documents just-in-time — bytes are staged, processed, returned and deleted within one request cycle — and the server database records nothing about any user's files, not even their names.

The application runs on a single AWS EC2 instance behind Caddy (automatic HTTPS), with managed external services for everything stateful: Neon (PostgreSQL) for accounts, Supabase Storage (S3) for transient staging, Brevo for transactional email and Google OAuth for sign-in. The instance itself holds no state and can be rebuilt at any time.

Property	Value
Live URL	https://pdftool4u.duckdns.org
User guide	/help (public, SEO-indexed, 13 step-by-step feature guides)
Hosting	AWS EC2 t3.micro (eu-north-1), Ubuntu, Docker Compose, Caddy HTTPS
Status monitoring	UptimeRobot pinging /api/health every 5 minutes
CI	GitHub Actions: typecheck (gate), tests, web build on every push to main
Readiness score	92/100 (Audit #3, July 2026)

2. Architecture

2.1 Monorepo layout

Package	Role
apps/web	Next.js 15 / React 19 front end. App router; proxies /api/* to the API so the browser sees a single origin (no CORS, first-party cookies).
apps/api	Express + TypeScript API. Auth, transient file staging, all PDF processing services.
packages/shared	Zod schemas + TypeScript types shared by client and server — one source of truth for every request/response contract.

2.2 Privacy-first data flow

The defining design. Originals live in the browser; the server is a stateless processor:

- **Library (client):** `local-store.ts` — IndexedDB v2 with a files store (metadata + Blob) and a private activity store. All list/view/rename/favorite/delete/download operations are local.
- **Tool runs (client orchestration):** `local-ops.ts` — stage (upload to server) → run (existing tool endpoint) → absorb (download result into IndexedDB) → cleanup (delete every server copy). Cleanup runs in `finally`, so failures cannot leave copies behind.
- **Defense in depth (server):** a retention job purges any staged file older than 60 minutes (covers browsers that disconnect mid-operation); stale guest accounts are purged after 24 h; the activity logger is a no-op — enforced by tests that assert the activity feed is always empty.

2.3 Technology stack

Layer	Technology
Front end	Next.js 15, React 19, TypeScript, TanStack Query, Radix UI, Tailwind CSS, framer-motion, next-themes (dark mode), pdfjs-dist (rendering), Inter via next/font
API	Express 4, TypeScript (tsx runtime), Zod validation, JWT + rotating hashed refresh tokens, Multer uploads, Helmet, express-rate-limit, compression
PDF engine	pdf-lib (+ @pdf-lib/fontkit for custom font embedding), pdfjs-dist, @napi-rs/canvas (rasterize), sharp (images), qpdf (encrypt/decrypt), archiver (ZIP)
Database	Neon PostgreSQL via Prisma 6 (driver adapters; libSQL locally)
Object storage	Supabase Storage, S3 protocol — transient staging only
Email	Brevo transactional HTTPS API (SMTP is blocked on many hosts)
Auth	Email+password (bcrypt), email verification, password reset, Google OAuth (one-time-code handoff), isolated guest accounts
Infra	Docker Compose (api, web, caddy), Caddy 2 auto-TLS, DuckDNS domain, AWS EC2 + 3 GB swap + 30 GB gp3

3. Feature Reference

Tool	Details
Upload	Drag & drop or picker; PDF/PNG/JPG/WebP up to 100 MB; magic-byte validation; single PDF auto-opens; page count computed client-side
View	pdf.js viewer with zoom and page navigation; password prompt for encrypted PDFs; inline preview for images
Edit	Text boxes, shapes, freehand ink, images, whiteout; click-to-replace existing text with matched size, weight, color (pixel-sampled) and closest font — granular Inter weights incl. Display optical variant for headlines; corner-handle resize (text scales font size, images scale uniformly)
Organize	Thumbnail grid: reorder, rotate, delete, insert blank pages; split ranges into separate files
Merge	Combine multiple PDFs in chosen order
Forms	Detect + fill interactive fields; optional flatten; create new form fields
Sign	Draw / type / upload signature; position + size; baked into the PDF
Redact	Draw boxes; content underneath is rasterized away permanently
Remove text	Strip exact text (e.g. watermarks) from every page
Watermark	Text watermark with size, rotation, opacity
Page numbers	6 positions, 3 formats, custom start number
Compress	Low/medium/high/custom (DPI + quality) with before/after size
Convert	PDF → PNG/JPG per page; images → single PDF
Protect / Unlock	qpdf AES encryption; in-viewer password entry; permanent unlock with known password
Download / ZIP	Single download from library; multi-select ZIP

Batch	Convert / compress / watermark many files at once
--------------	---

Accounts: registered users (email verification, password reset, Google sign-in) and throwaway guest accounts (auto-created, isolated, purged after 24 h). Activity feed and storage meter are browser-local.

4. Infrastructure & Deployment

4.1 Production topology

Component	Detail
EC2 instance	t3.micro (1 GB RAM), Ubuntu, eu-north-1; 3 GB swapfile (required for builds); 30 GB gp3 root volume
Containers	api (Express, port 4000), web (Next.js, port 3000), caddy (80/443) — docker-compose.oracle.yml, restart: unless-stopped
TLS	Caddy auto-provisions/renews Let's Encrypt for pdftool4u.duckdns.org
DNS	DuckDNS free subdomain → instance public IP (update on IP change)
Security group	Inbound: 22 (SSH, restricted), 80, 443
Env config	.env on the box (from .env.oracle.example): URLs, Neon, JWT secrets, Supabase S3, Brevo, Google OAuth

4.2 Deploy / update runbook

```
cd ~/pdf_tool && git pull && docker compose -f docker-compose.oracle.yml up -d --build
```

Build takes 10–20 min on 1 GB (swap carries the Next.js build). Logs: `docker compose -f docker-compose.oracle.yml logs -f`. Config-only change: `up -d --force-recreate (no build)`.

4.3 CI pipeline

GitHub Actions on every push/PR to main: fresh Linux dependency install (committed lockfile is Windows-generated and omits Linux natives), Prisma generate, typecheck (hard gate), API test suites (node:test), web production build. Lint is advisory.

4.4 Local development

```
npm install && npm run dev # web :3000 + api :4000, SQLite/libSQL local DB
```

Local runs use `STORAGE_DRIVER=local` and console-logged email; Google OAuth and Inter font embedding activate only where configured/installed (graceful fallbacks).

5. Operations & Maintenance

5.1 Monitoring

- UptimeRobot: HTTP check on `/api/health` every 5 min with email alert.
- Health endpoint sits above the rate limiter; request log lines on the API (skips health checks).

5.2 Scheduled jobs (in-process)

- Expired token purge — every 12 h.
- Data retention purge — every 15 min: deletes staged files older than 60 min and restores quota (safety net for interrupted client cleanups).
- Stale guest purge — guests older than 24 h removed on new guest login.

5.3 Security posture

- Helmet CSP (default-src 'none'; dev-only 'unsafe-eval' for Next.js Fast Refresh), CORS locked to the web origin, tiered rate limits (300/5min general, 10/15min credentials).
- Uploads validated by magic bytes before storage; JWT secrets ≥ 32 chars enforced at boot; refresh tokens hashed + rotated in httpOnly Secure cookies.
- All production secrets rotated July 2026 after setup exposure; GitHub PATs revoked.

5.4 Known limitations / open items

- Single-instance deployment — no redundancy (acceptable at current scale; rebuild runbook exists).
- Containers run as root (planned: non-root USER).
- Old Render deployment must remain suspended; EC2 free tier ends after 12 months (~\$8–10/mo after).
- Browser-local files are per-device: clearing site data deletes them; no cross-device sync (documented in UI).
- Settings still shows a legacy activity-logging toggle (server logging is permanently off).
- No E2E test suite yet; font fidelity for proprietary brand fonts is substitution-based (industry ceiling).

6. Release History (July 2026)

Release	Contents
Security hardening	6 audit fixes: CSP, rate limits, magic-byte sniffing, token rotation, quota, config validation
Managed services	Neon PostgreSQL, Supabase S3 storage, Brevo email, Google OAuth (proxy-safe one-time-code handoff)
Feature wave	Page numbers, ZIP download, password protect/unlock (qpdf), e-signatures; OCR/Office conversion removed for footprint
UX wave	Dark mode, branded 404, quick-tools grid, auto-open viewer, retention banner
EC2 migration	Render → AWS EC2 with Caddy auto-HTTPS, DuckDNS domain, swap + disk expansion; UptimeRobot monitoring
Privacy architecture	Browser-local IndexedDB library, just-in-time stateless processing, server activity logging removed (test-enforced), browser-local activity + storage meter
UI polish	Upload-first dashboard, tool-card file picker, dismissible privacy banner, guest badge, neutral progress track, viewer Tools dropdown
Viewer fixes	Password prompt for encrypted PDFs; inline image preview
Editor fidelity	Replacement text matches size, weight (real font data), color (pixel sampling); Inter family embedding with granular weights + Display variant; bold/italic serif and oblique sans; corner-handle resize

7. Roadmap (Suggested)

- **P1:** Elastic IP; non-root containers; remove legacy activity toggle; sitemap + Google Search Console for /help.
- **P2:** Offline PWA (natural fit for browser-local files); mobile polish pass; Playwright E2E suite; Linux-committed lockfile.
- **P3:** Opt-in AI features (chat/summarize/extract) with explicit consent — design must respect the privacy promise; custom licensed-font drop-in for perfect brand fidelity.

PDF Tool Technical Documentation — July 2026 edition. Supersedes PDF-Tool-Documentation.pdf (pre-migration).